Replicação do *Reflective Gaussian Splatting* e Propostas para a Modelagem de Visibilidade em Superfícies Reflexivas

João Pedro Maciel Pimenta
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brasil
jpmp3@cin.ufpe.br

Francisco Paulo Magalhães Simões
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brasil
fpms@cin.ufpe.br

***Resumo*—A reconstrução de superfícies reflexivas continua sendo um ponto fraco dos métodos de síntese de novas vistas baseados em Gaussian Splatting, pois a aparência dependente da direção de observação é aprendida sem um modelo físico explícito da luz refletida. O Reflective Gaussian Splatting (Ref-Gaussian) aborda o problema com renderização diferida fisicamente baseada sobre Gaussianas 2D e com um termo de inter-reflexão calculado por ray tracing sobre uma malha extraída periodicamente da cena. Este relatório descreve o método, identifica suas limitações quanto à aproximação de visibilidade e ao custo da extração de malha, e propõe duas linhas de investigação para a dissertação. Como primeiro passo experimental, o código oficial foi portado e executado em uma GPU de notebook sob Windows, e o baseline foi replicado no conjunto Shiny Blender Synthetic. Nas seis cenas, a média fica em PSNR de 34,46 dB, SSIM de 0,973 e LPIPS de 0,057, contra 35,04 dB, 0,973 e 0,056 no artigo original. A avaliação de checkpoints intermediários mostra que boa parte do desvio vem do protocolo: o modelo piora nas últimas iterações e o script de avaliação usa a malha da última iteração; com 40 mil iterações, as duas cenas mais difíceis chegam ao nível do artigo. A análise do mapa de visibilidade mostra que os erros se concentram nos pixels em que o raio refletido é dado como bloqueado, e que a causa principal é a radiância indireta, aprendida por Gaussiana, e não o teste de oclusão.**

***Index Terms*—Gaussian Splatting, superfícies reflexivas, renderização fisicamente baseada, inter-reflexão, síntese de novas vistas.**

I. Introdução

O 3D Gaussian Splatting (3DGS) [1] representa uma cena como um conjunto de Gaussianas anisotrópicas rasterizadas de forma diferenciável, alcançando qualidade comparável à dos campos de radiância neurais com renderização em tempo real. Sua aparência dependente de vista, porém, é modelada por harmônicos esféricos de baixa ordem, o que é insuficiente para superfícies especulares: reflexos nítidos são aprendidos como geometria falsa "atrás" do objeto e as normais resultantes ficam incorretas.

Trabalhos recentes incorporam modelos de sombreamento ao 3DGS. O GaussianShader [4] associa a cada Gaussiana uma função de sombreamento simplificada; o 3DGS-DR [5] adia o cálculo da reflexão para o espaço da imagem; e o Ref-Gaussian [3], adotado aqui como *baseline*, combina Gaussianas 2D [2], renderização diferida fisicamente baseada e um termo de inter-reflexão. O Ref-Gaussian supera os métodos anteriores no Shiny Blender [6] em todas as cenas (Tabela I).

Tabela I
PSNR no Shiny Blender reportado por Yao et al. [3] para métodos anteriores, como contexto para o baseline.

Método	Ball	Car	Coffee	Helmet	Teapot	Toaster
3DGS [1]	27,65	27,26	32,30	28,22	45,71	20,99
2DGS [2]	25,97	26,38	32,31	27,42	44,97	20,42
Ref-NeRF [6]	33,16	30,44	33,99	29,94	45,12	26,12
GShader [4]	30,99	27,96	32,39	28,32	45,86	26,28
3DGS-DR [5]	33,43	30,48	34,53	31,44	47,04	26,76
Ref-Gaussian [3]	**37,01**	**31,04**	**34,63**	**32,32**	**47,16**	**28,05**

Este relatório tem dois objetivos: (i) descrever o método base e as linhas de investigação propostas para a dissertação (Seção II); e (ii) replicar os resultados do baseline em hardware de consumo, condição necessária antes de propor modificações (Seções III e IV).

II. Solução Proposta

A. Método base: Reflective Gaussian Splatting

A Fig. 1 resume o pipeline do Ref-Gaussian. Seus componentes são descritos a seguir.

Representação. A cena é composta por *surfels* Gaussianos 2D [2], discos orientados definidos por centro, dois vetores tangentes, escalas e opacidade. A normal de cada primitiva é o vetor perpendicular ao seu plano, o que fornece geometria de superfície mais precisa que a das Gaussianas 3D. Cada primitiva carrega ainda os atributos de material do modelo Disney/PBR: albedo $\mathbf{a}$, metalicidade m e rugosidade r.

Renderização diferida. Em vez de sombrear cada Gaussiana e depois combinar as cores, os atributos ($\mathbf{a}$, m, r, normal $\mathbf{n}$ e profundidade) são primeiro combinados por *alpha blending* em mapas por pixel. O sombreamento é então avaliado uma única vez por pixel, o que gera reflexos mais nítidos e

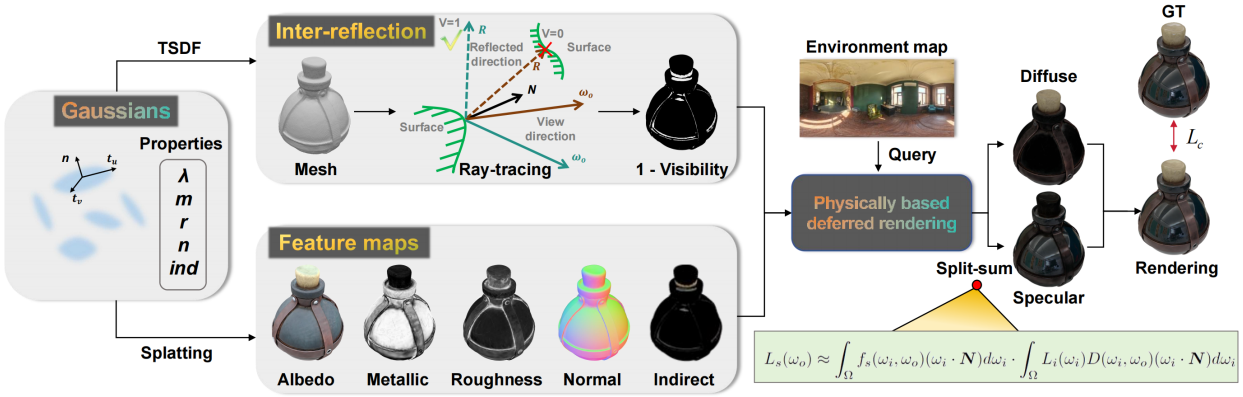


Figura 1. Pipeline do Ref-Gaussian: Gaussianas 2D com atributos de material são rasterizadas em mapas por pixel (renderização diferida); o sombreamento combina iluminação direta de um mapa de ambiente com inter-reflexão fundamentada nas próprias Gaussianas. Fonte: Yao et al. [3].

Tabela II
REPLICAÇÃO DO REF-GAUSSIAN NO SHINY BLENDER SYNTHETIC: VALORES REPORTADOS NO ARTIGO [3] E OBTIDOS NESTE TRABALHO (RTX 4050 LAPTOP); Δ É A DIFERENÇA DE PSNR DA REPLICAÇÃO EM RELAÇÃO AO ARTIGO. MODELO FINAL (50 MIL ITERAÇÕES), AVALIADO COM O `eval.py` OFICIAL.

Métrica	Origem	Ball	Car	Coffee	Helmet	Teapot	Toaster	Média
PSNR ↑	Artigo	37,01	31,04	34,63	32,32	47,16	28,05	35,04
	Replicação	36,07	31,25	34,29	30,45	47,19	27,51	34,46
	Δ	-0,94	+0,21	-0,34	-1,87	+0,03	-0,54	-0,57
SSIM ↑	Artigo	0,981	0,964	0,976	0,971	0,998	0,948	0,973
	Replicação	0,985	0,966	0,976	0,965	0,998	0,948	0,973
LPIPS ↓	Artigo	0,098	0,033	0,076	0,049	0,006	0,074	0,056
	Replicação	0,089	0,031	0,077	0,061	0,006	0,074	0,057

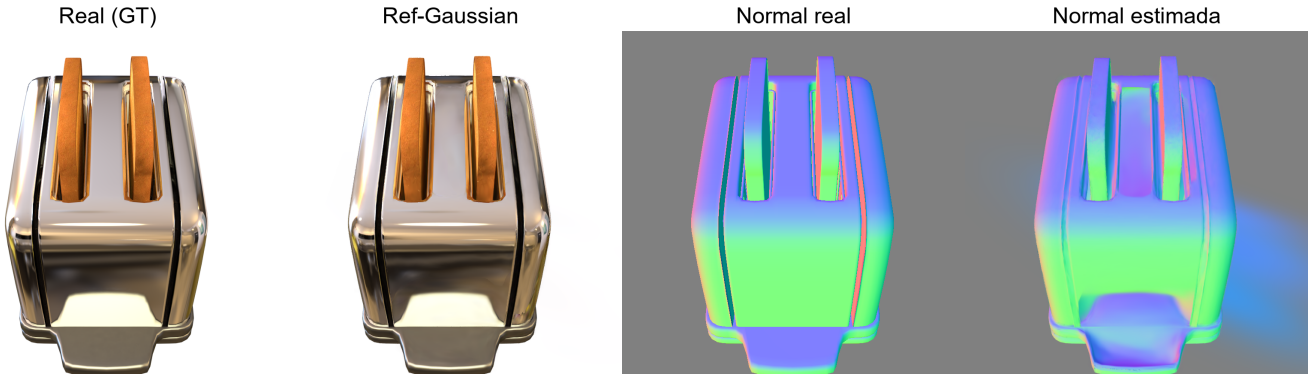


Figura 2. Resultado qualitativo na cena *toaster* (vista de teste 0) após 50.000 iterações. Da esquerda para a direita: imagem real, renderização do Ref-Gaussian, normal real e normal estimada.

consistentes. A radiância de saída na direção $\boldsymbol{\omega}_o$ é dividida em um termo difuso e um especular:

$$L(\boldsymbol{\omega}_o) = L_d + L_s, \quad L_d = (1-m)\,\mathbf{a}\,\frac{1}{\pi}\int_\Omega L_i(\boldsymbol{\omega}_i)\,(\boldsymbol{\omega}_i\cdot\mathbf{n})\,d\boldsymbol{\omega}_i. \tag{1}$$

O termo especular usa a aproximação *split-sum* [8]:

$$L_s \approx \left(\mathbf{F}_0\,A(\mathbf{n}\cdot\boldsymbol{\omega}_o, r) + B(\mathbf{n}\cdot\boldsymbol{\omega}_o, r)\right)\,\tilde{L}(\boldsymbol{\omega}_r, r), \tag{2}$$

com $\mathbf{F}_0 = 0{,}04(1-m) + m\,\mathbf{a}$, termos A e B pré-integrados da BRDF em uma tabela, e $\tilde{L}$ a iluminação pré-filtrada amostrada na direção de reflexão $\boldsymbol{\omega}_r$ em um mapa de ambiente cúbico treinável com níveis de mipmap indexados pela rugosidade.

Inter-reflexão fundamentada em Gaussianas. O mapa de ambiente representa apenas luz vinda do infinito; reflexos de uma parte do objeto sobre outra exigem um termo indireto. O Ref-Gaussian separa a componente especular em

$$L_s = V\,L_s^{\text{dir}} + (1-V)\,L_s^{\text{ind}}, \qquad V \in \{0,1\}, \tag{3}$$

em que a visibilidade V é obtida lançando um único raio na direção de reflexão $\boldsymbol{\omega}_r$ contra uma malha extraída das Gaussianas por fusão TSDF. A radiância indireta L_s^{ind} não é

buscada no ponto atingido pelo raio: no código oficial, cada Gaussiana carrega um atributo de radiância indireta próprio, representado por harmônicos esféricos de grau 3 avaliados na direção de reflexão (ou, opcionalmente, gaussianas esféricas anisotrópicas), rasterizado como os demais atributos e ajustado apenas pela perda fotométrica. A malha é reextraída periodicamente durante o treino.

Propagação de normais sensível ao material. Para corrigir normais em regiões altamente especulares, as Gaussianas com $m \geq 0{,}02$ e $r \leq 0{,}1$ têm suas escalas periodicamente ampliadas, propagando normais corretas para as vizinhas.

Treinamento em estágios. O treino começa com sombreamento por Gaussiana, que estabiliza a geometria, e depois passa à renderização diferida. A função de perda é

$$\mathcal{L} = \mathcal{L}_c + \lambda_n \mathcal{L}_n + \lambda_s \mathcal{L}_{\text{smooth}}, \tag{4}$$

com $\mathcal{L}_c$ a perda fotométrica, $\mathcal{L}_n$ a consistência entre normais renderizadas e derivadas da profundidade [2] e $\mathcal{L}_{\text{smooth}}$ uma suavização de normais. O artigo usa $\lambda_n = 0{,}05$ e $\lambda_s = 1{,}0$.

B. Limitações identificadas

1) **Visibilidade pontual.** A Eq. (3) testa a oclusão apenas na direção de reflexão especular perfeita e usa esse único teste para todo o lóbulo especular. Os próprios autores apontam que isso introduz ruído na estimativa de materiais. O erro tende a crescer com a rugosidade, quando o lóbulo se alarga.
2) **Dependência de malha extraída.** O ray tracing exige reconstruir uma malha por TSDF a cada poucos milhares de iterações e reconstruir a estrutura de aceleração (BVH). Isso adiciona custo, cria uma segunda representação geométrica que pode divergir das Gaussianas e torna a visibilidade não diferenciável em relação a elas.
3) **Escopo.** O método é desenhado para cenas estáticas e predominantemente especulares; os autores reconhecem desempenho inferior em cenas não especulares.

C. Linhas de investigação propostas

As propostas a seguir são hipóteses de trabalho para a dissertação e serão avaliadas contra o baseline replicado neste relatório.

P1 – Visibilidade sobre o lóbulo especular. Substituir o teste binário único por uma estimativa da visibilidade média sobre o lóbulo da BRDF:

$$\bar{V} \approx \frac{1}{K} \sum_{k=1}^{K} V(\boldsymbol{\omega}_k), \quad \boldsymbol{\omega}_k \sim D_{\text{GGX}}(\,\cdot\,; \boldsymbol{\omega}_r, r), \tag{5}$$

com poucas direções $\boldsymbol{\omega}_k$ obtidas por amostragem por importância da distribuição GGX, de forma estratificada e com K adaptativo à rugosidade ($K = 1$ recupera o baseline em superfícies espelhadas). Para manter o custo baixo, serão avaliadas reutilização temporal das amostras entre iterações e aproximações por cone. A avaliação usará PSNR/SSIM/LPIPS, o erro de albedo e rugosidade nas cenas sintéticas com material conhecido, e uma ablação de K contra tempo de treino.

P2 – Visibilidade direta a partir das primitivas. Eliminar a extração de malha calculando a oclusão diretamente sobre as Gaussianas, acumulando a transmitância ao longo do raio refletido com uma estrutura de aceleração sobre envelopes das primitivas, na linha do ray tracing de Gaussianas [9]. Além de remover o custo do TSDF, essa formulação permite uma visibilidade contínua e potencialmente diferenciável em relação à geometria. Com o ponto de interseção disponível, a radiância indireta pode ser obtida das próprias Gaussianas atingidas, em vez de um atributo aprendido à parte; a Seção IV mostra que esse atributo é a principal fonte de erro observada. As métricas são as mesmas de P1, mais o tempo total de treino e o consumo de memória.

P3 – Extensão dinâmica (trabalho futuro). Estender a representação por Gaussianas 2D a objetos reflexivos em movimento, tratando a coerência temporal dos reflexos. Esta linha depende dos resultados de P1/P2 e fica como perspectiva.

III. Experimentos

O objetivo desta etapa é replicar o baseline com o código oficial antes de qualquer modificação.

A. Ambiente e reprodutibilidade

Os experimentos usam o código oficial do Ref-Gaussian (commit `af7691a`, mar. 2025) em um notebook com GPU NVIDIA GeForce RTX 4050 Laptop (6 GB), Windows 11, CUDA 11.8, PyTorch 2.0.0 e Python 3.8. O código foi desenvolvido para Linux, e as extensões CUDA (rasterizador de *surfels*, *simple-knn*, codificador de cubemap e o ray tracer baseado em BVH) precisaram de ajustes para compilar com o MSVC 14.44 (Visual Studio 2022), mais recente que o oficialmente suportado pelo CUDA 11.8. Os principais ajustes foram: permitir o compilador não suportado; incluir explicitamente cabeçalhos que o GCC inclui de forma transitiva; ativar o modo de conformidade do MSVC (`/permissive-`, `/Zc:__cplusplus`), necessário para compilar a biblioteca Eigen junto com o nvcc; e impedir a inclusão do pybind11 nos arquivos CUDA. Nenhum desses ajustes altera o código-fonte do método; todos foram feitos por meio de flags de compilação, registradas em scripts no repositório do projeto.

B. Conjunto de dados

Nesta etapa usa-se o Shiny Blender Synthetic [6], com seis objetos sintéticos de materiais altamente especulares (*ball*, *car*, *coffee*, *helmet*, *teapot* e *toaster*). Cada cena tem 100 vistas de treino e 200 de teste, em resolução 800×800 e fundo branco. Os conjuntos Glossy Synthetic [7] e Shiny Blender Real serão usados nas próximas etapas.

C. Protocolo

O treino segue os argumentos do script `train.sh` do repositório oficial, com a configuração padrão do código: 50.000 iterações, sombreamento por Gaussiana até a iteração 18.000, inter-reflexão a partir da iteração 20.000, extração de malha (resolução 512) a cada 2.000 iterações e densificação até a iteração 25.000. As cenas *helmet* e *ball* usam $\lambda_s = 1{,}0$,

Tabela III
EFEITO DO CHECKPOINT E DA MALHA USADA NA VISIBILIDADE, NAS MESMAS 200 VISTAS DE TESTE. O `eval.py` OFICIAL USA AS GAUSSIANAS E A MALHA DA ÚLTIMA ITERAÇÃO (50K).

Cena	Gaussianas / malha	PSNR	SSIM	LPIPS
Toaster	30k / 30k	27,60	0,949	0,071
	40k / 40k	**28,08**	**0,951**	**0,068**
	50k / 48k	27,94	0,951	0,071
	50k / 50k (`eval.py`)	27,51	0,948	0,074
	Artigo	28,05	0,948	0,074
Helmet	30k / 30k	31,50	0,970	0,053
	40k / 40k	**32,09**	**0,971**	**0,050**
	50k / 48k	31,24	0,968	0,054
	50k / 50k (`eval.py`)	30,45	0,965	0,061
	Artigo	32,32	0,971	0,049

como no script oficial. Esse cronograma difere do descrito no artigo (18.000 iterações iniciais mais 40.000 com renderização diferida, extração a cada 3.000 passos); optou-se pela configuração publicada no código.

As métricas são as do artigo, calculadas pelo script de avaliação oficial sobre as 200 vistas de teste: PSNR, SSIM e LPIPS (VGG). Reportam-se também a taxa de renderização (FPS) e o tempo de treino.

IV. RESULTADOS E DISCUSSÃO

Métricas. A Tabela II compara os resultados obtidos com os do artigo nas seis cenas. Na média, o SSIM (0,973) coincide com o reportado, o LPIPS difere em 0,001 (0,057 contra 0,056) e o PSNR fica 0,57 dB abaixo (34,46 contra 35,04 dB). Por cena, *car* (+0,21 dB) e *teapot* (+0,03 dB) ficam acima do artigo nas três métricas ou empatam; *coffee* ($-0{,}34$ dB) e *toaster* ($-0{,}54$ dB) ficam próximas, com SSIM e LPIPS praticamente iguais; quase todo o déficit vem de *helmet* ($-1{,}87$ dB) e *ball* ($-0{,}94$ dB, mas com SSIM e LPIPS melhores que os do artigo). Diferenças desse porte são esperadas entre execuções de métodos com treino estocástico, em outra GPU e com outro compilador, e há também a diferença de cronograma de treino descrita na Seção III; a avaliação de checkpoints a seguir mostra que parte delas vem do protocolo. Em quatro cenas (*ball*, *car*, *teapot* e *toaster*) o resultado supera o melhor método anterior segundo o artigo (3DGS-DR); em *coffee* e *helmet* fica abaixo dele.

Checkpoint e malha de avaliação. As curvas de treino (Fig. 3) mostram que, nas cenas com maior desvio, o PSNR de teste atinge um pico por volta de 43 mil iterações e depois cai: na *toaster* o pico é de 28,19 dB e, na *helmet*, de 32,29 dB, praticamente os valores do artigo. Para separar os efeitos, os checkpoints salvos foram avaliados com diferentes malhas (Tabela III). Dois efeitos aparecem. Primeiro, o modelo de 40 mil iterações é melhor que o final: supera o artigo nas três métricas na *toaster* e fica a 0,23 dB dele na *helmet*. Segundo, o `eval.py` oficial carrega a malha extraída na última iteração, ao passo que a medição feita durante o treino usava a malha da iteração 48 mil; só essa troca custa 0,43 dB na *toaster* e 0,79 dB na *helmet*, e explica a diferença entre o PSNR

Tabela IV
CUSTO NA RTX 4050 LAPTOP: TEMPO DE TREINO (50.000 ITERAÇÕES) E TAXA DE RENDERIZAÇÃO NA AVALIAÇÃO.

Cena	Tempo de treino	FPS
ball	1h25min	10,6
car	1h22min	57,3
coffee	1h08min	61,2
helmet	1h16min	52,3
teapot	0h49min	74,8
toaster	1h46min	38,9

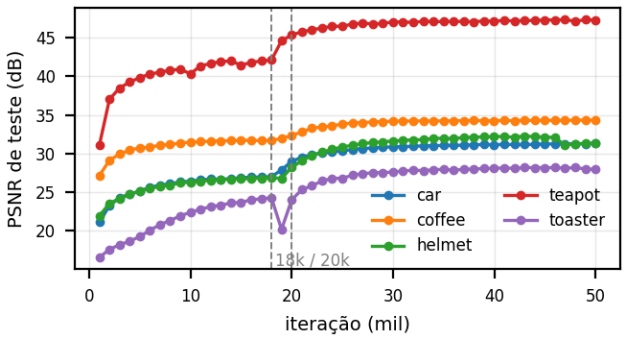


Figura 3. PSNR nas vistas de teste ao longo do treino, para as cenas com log de treino registrado (todas exceto *ball*). As linhas tracejadas marcam o fim do sombreamento por Gaussiana (18 mil iterações) e o início da inter-reflexão (20 mil).

medido pelo treino e pelo script de avaliação. Os números do artigo são, portanto, praticamente alcançáveis com o código publicado, e a escolha do checkpoint e da malha deve ser fixada no protocolo das próximas comparações.

Dinâmica do treino. A Fig. 3 mostra a evolução do PSNR de teste, medido pelo próprio script de treino a cada mil iterações. Na cena *toaster*, o estágio de sombreamento por Gaussiana leva o PSNR a cerca de 24,3 dB na iteração 18 mil. A troca para a renderização diferida derruba o valor temporariamente para 20,2 dB, já que os mapas de material passam a ser combinados antes do sombreamento, mas ele se recupera em menos de 2 mil iterações. A partir da inter-reflexão (20 mil), o PSNR sobe para cerca de 28,2 dB em torno de 43 mil iterações; esse ganho de quase 4 dB após a etapa inicial indica que a renderização diferida e o termo indireto respondem por boa parte da qualidade final. Nas últimas milhares de iterações há uma leve queda (de 28,19 para 27,96 dB), mais forte na *helmet* (de 32,29 para 31,36 dB); nas cenas *coffee*, *teapot* e *car* a curva só se estabiliza. A diferença entre o valor medido no treino e o do `eval.py` é explicada pela malha de avaliação, como mostrado acima.

Custo computacional. A Tabela IV resume o custo por cena. O treino levou de 49 minutos (*teapot*) a 1h46min (*toaster*), e a renderização na avaliação ficou entre 39 e 75 FPS, exceto na *ball*. O artigo reporta 0,58 h de treino em média e 122 FPS em uma NVIDIA A6000. A diferença é coerente com a distância de capacidade entre uma GPU de notebook

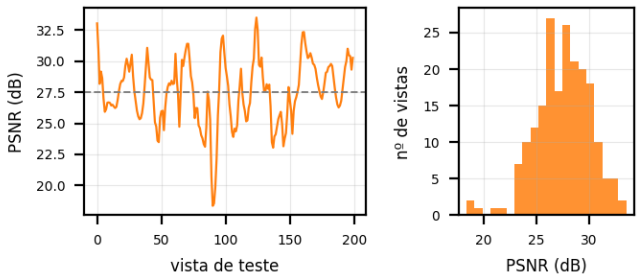


Figura 4. PSNR por vista de teste na cena *toaster*: valor de cada uma das 200 vistas (esquerda, linha tracejada na média) e histograma (direita).

de 6 GB e uma GPU de estação de trabalho de 48 GB. Ainda assim, mostra que o baseline pode ser treinado integralmente em hardware de consumo, o que viabiliza os experimentos da dissertação. A *ball* treinou mais rápido (1h25min), coerente com seu modelo menor (38 mil Gaussianas e malha de 76 mil vértices, contra 83 mil e 126 mil na *toaster*). Seu FPS na avaliação (10,6), porém, foi menor apesar do modelo menor, o que indica interferência do estado da máquina no momento da medição, como modo de energia ou outros processos na GPU. As medições de FPS serão repetidas em condições controladas antes de qualquer comparação de desempenho.

Análise qualitativa. Na Fig. 2, a renderização reproduz os reflexos especulares do corpo metálico, incluindo os reflexos das fatias de pão sobre a carcaça, uma situação de inter-reflexão. O mapa de normais estimado, por outro lado, é mais suave que o real: os frisos verticais estreitos nas laterais praticamente desaparecem, e surgem artefatos de baixa opacidade no fundo, fora do objeto. Esse descompasso entre aparência correta e geometria aproximada é relevante para as propostas da Seção II. Como a visibilidade do baseline é calculada sobre uma malha derivada dessa geometria, erros finos de normal e de superfície se propagam para o termo de inter-reflexão. Esse efeito motiva tanto a amostragem do lóbulo (P1) quanto a visibilidade calculada diretamente sobre as primitivas (P2).

Análise por vista. A média de 27,51 dB esconde uma variação grande entre vistas (Fig. 4). O PSNR por vista tem desvio-padrão de 2,58 dB e vai de 18,36 a 33,50 dB; 32 das 200 vistas ficam abaixo de 25 dB. A curva ao longo das vistas, que percorrem uma trajetória em torno do objeto, mostra que as quedas se concentram em faixas contíguas de ângulos, e não em vistas isoladas. Isso indica que o erro depende da geometria de observação, ou seja, de quais regiões do ambiente cada superfície reflete naquela vista. A Fig. 5 compara os extremos. Na melhor vista, a face frontal reflete um ambiente suave e o erro fica restrito às bordas. Na pior, a lateral espelhada da torradeira, que na imagem real reflete uma região escura próxima, é renderizada com um reflexo claro e mal posicionado, e o erro se concentra nessa área. Entre as outras cenas, só a *helmet* tem quedas comparáveis (mínimo de 19,4 dB e 24 vistas abaixo de 25 dB, com o erro concentrado na viseira espelhada); *car* tem 5 vistas abaixo de 25 dB e *ball*, *coffee* e *teapot* nenhuma abaixo de 29 dB.

Teste da hipótese de visibilidade. Para verificar a origem dessas falhas, o mapa de visibilidade, a luz direta e a radiância indireta foram exportados nas 200 vistas de teste da *toaster* (Fig. 7). Três resultados se destacam. (i) O erro se concentra onde o raio é dado como bloqueado: o erro absoluto médio nos pixels com $V = 0$ é 2,2 vezes o dos pixels com $V = 1$ (0,054 contra 0,024), e 4,3 vezes na pior vista (0,270 contra 0,063). (ii) A quantidade de oclusão por vista quase não prevê a qualidade (correlação de 0,11 entre a fração de pixels com $V = 0$ e o PSNR); importa onde a oclusão ocorre, e não quanto. (iii) Na pior vista, a luz direta do mapa de ambiente já reproduz o tom escuro da face direita, mas uma região irregular dessa face recebe $V = 0$ e passa a usar a radiância indireta, clara e ruidosa, o que produz o erro. A hipótese inicial, de um raio que deveria ser bloqueado e escapa, estava errada: o erro aparece depois do bloqueio. Para descartar uma autointerseção numérica (o raio parte exatamente do ponto da superfície, sem deslocamento), a mesma avaliação foi repetida deslocando a origem do raio em até 0,05 unidade, cerca de 1% do raio da cena: o PSNR médio passou de 27,51 para 27,54 dB e a fração ocluída de 22,2% para 21,3%, com a mancha de $V = 0$ preservada. As oclusões são, portanto, consistentes com a malha; o que falha é a radiância indireta, um atributo em harmônicos esféricos aprendido por Gaussiana e não a luz do ponto atingido. Isso desloca o foco das propostas: a transição binária de V é o que P1 suaviza, mas a fonte principal de erro observada é o termo que P2 pode substituir.

Decomposição de material e iluminação. A renderização diferida permite inspecionar separadamente os fatores que o modelo aprendeu (Fig. 8). A separação física é em grande parte coerente: o corpo da torradeira recebe metalicidade alta e rugosidade baixa, e as fatias de pão recebem metalicidade baixa e rugosidade alta, de modo que a componente especular fica concentrada no metal e a difusa no pão. A decomposição, porém, não é limpa. O albedo da região entre as fatias apresenta cores espúrias, azuladas e iridescentes, que não existem no objeto; a rugosidade varia em manchas sobre o metal, que deveria ser uniforme; e todos os mapas mostram um halo de Gaussianas de baixa opacidade fora do objeto, à esquerda. Os autores atribuem justamente à aproximação de visibilidade parte do ruído na estimativa de materiais, e esta observação dá uma forma qualitativa e mensurável a essa limitação: as métricas de imagem quase não são afetadas, já que o erro final é baixo, mas os materiais recuperados são menos confiáveis do que a renderização sugere. Para aplicações que dependem dos materiais, como reiluminação e inserção de objetos em realidade aumentada, esse é o aspecto mais relevante a melhorar. A Fig. 9 mostra o mapa de ambiente aprendido. O modelo reconstrói, apenas a partir dos reflexos, um ambiente interno reconhecível, com janelas, luminárias, uma cama e uma poltrona, embora borrado e com distorções nas regiões pouco vistas pelos reflexos, como a parte inferior do mapa.

Cena convexa versus côncava. A cena *ball* (Fig. 6) funciona como um controle natural para a análise da *toaster*.

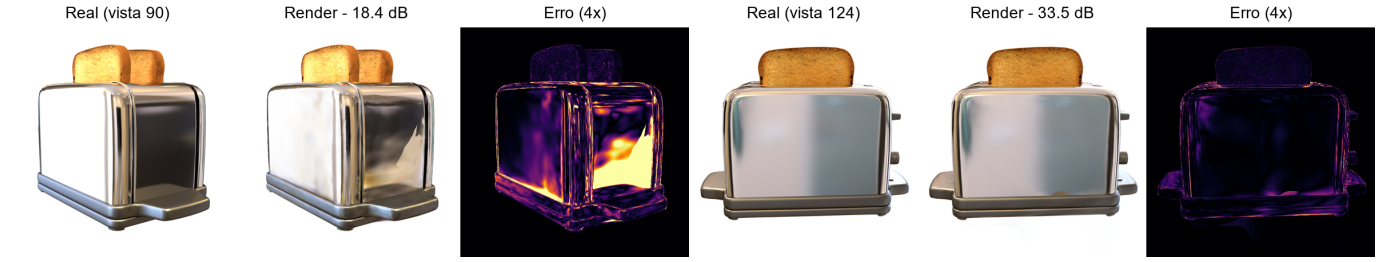


Figura 5. Pior (vista 90, 18,4 dB) e melhor (vista 124, 33,5 dB) vistas de teste da cena *toaster*, com o erro absoluto amplificado 4 vezes. Na pior vista, a lateral espelhada, que na imagem real reflete uma região escura, é renderizada clara.

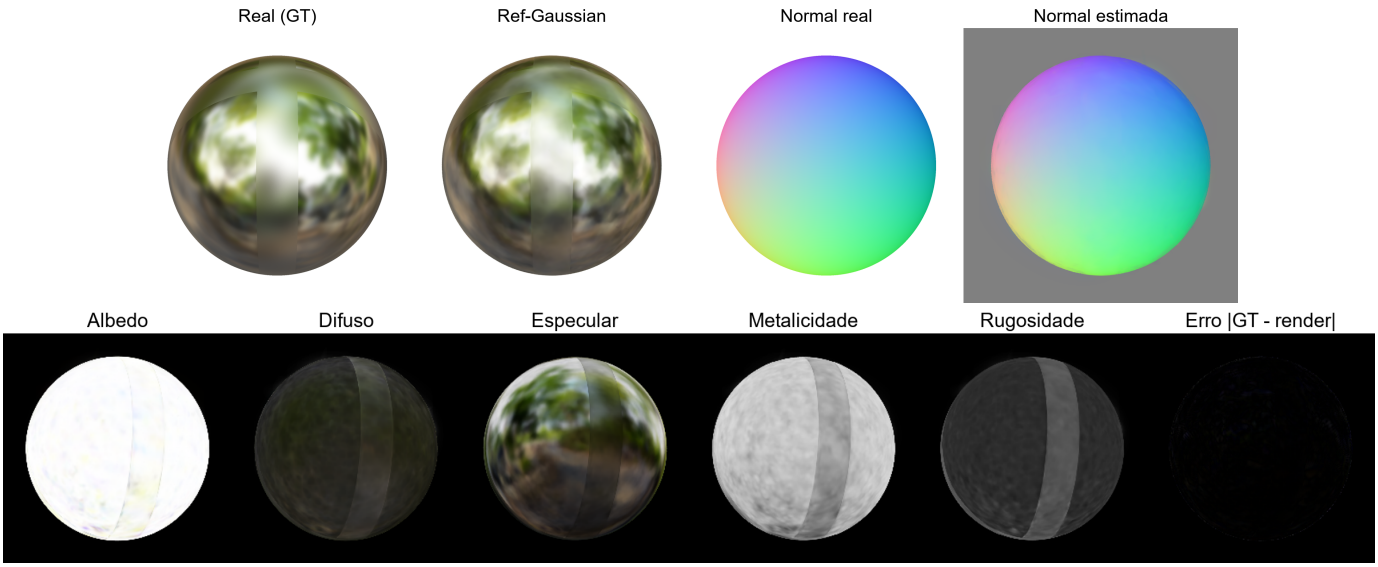


Figura 6. Cena *ball*, convexa e sem inter-reflexões. Acima: imagem real, renderização, normal real e normal estimada (vista de teste 0). Abaixo: mapas intermediários da renderização diferida (vista de treino). A faixa mais rugosa da esfera aparece clara na rugosidade e mais escura na metalicidade.

Por ser uma esfera, uma superfície convexa, nenhum ponto reflete outra parte do próprio objeto: a visibilidade ao longo de qualquer direção de reflexão é sempre 1, e a aproximação da Eq. (3) é exata. Os sintomas observados na *toaster* praticamente desaparecem. O PSNR por vista tem desvio-padrão de 1,90 dB, contra 2,58 dB na *toaster*, e o mínimo é de 31,66 dB, sem nenhuma vista abaixo de 25 dB. As normais estimadas coincidem visualmente com as reais, e a decomposição recupera a estrutura do material: a faixa mais rugosa que atravessa a esfera aparece com rugosidade maior e metalicidade menor, e a componente especular reflete o ambiente com nitidez. O ruído não some totalmente, pois a metalicidade ainda apresenta manchas sobre uma superfície que deveria ser uniforme, mas é bem menor que o da *toaster*. Com as seis cenas, o quadro se mantém, com uma ressalva: a *teapot*, quase convexa, também não tem quedas (mínimo de 40,3 dB), mas a *coffee*, que tem o interior côncavo da xícara, também não, provavelmente porque a cerâmica é menos espelhada que o metal. As quedas fortes ficam restritas a *toaster* e *helmet*, objetos metálicos ou espelhados com partes que se refletem. As diferenças entre cenas não se devem só à geometria (materiais e ambientes também diferem), mas o padrão é consistente com o teste de visibilidade acima.

Síntese. Os resultados desta etapa indicam que (i) o baseline é reproduzível em hardware de consumo nas seis cenas do Shiny Blender, com SSIM médio igual ao do artigo, LPIPS médio a 0,001 dele e um déficit de PSNR explicado em boa parte pela escolha do checkpoint e da malha de avaliação; (ii) a qualidade média esconde falhas localizadas, concentradas em duas cenas metálicas com partes que se refletem; (iii) essas falhas ocorrem nos pixels em que o método usa a radiância indireta, e não por oclusões numericamente erradas; e (iv) os materiais recuperados são ruidosos mesmo quando a imagem renderizada é fiel. Os pontos (iii) e (iv) apontam o termo indireto como o alvo principal, o que reforça P2 e dá a P1 um papel complementar.

Próximos passos. (i) Fixar no protocolo a avaliação com o checkpoint e a malha correspondentes e repetir a Tabela III nas seis cenas; (ii) repetir o teste de visibilidade na *helmet* e medir, em todas as cenas, a fração do erro nos pixels com $V = 0$; (iii) estender a replicação ao Glossy Synthetic e ao Shiny Blender Real; (iv) medir separadamente o custo da extração de malha e do ray tracing; e (v) prototipar P2 buscando a radiância indireta nas Gaussianas do ponto atingido, comparando com

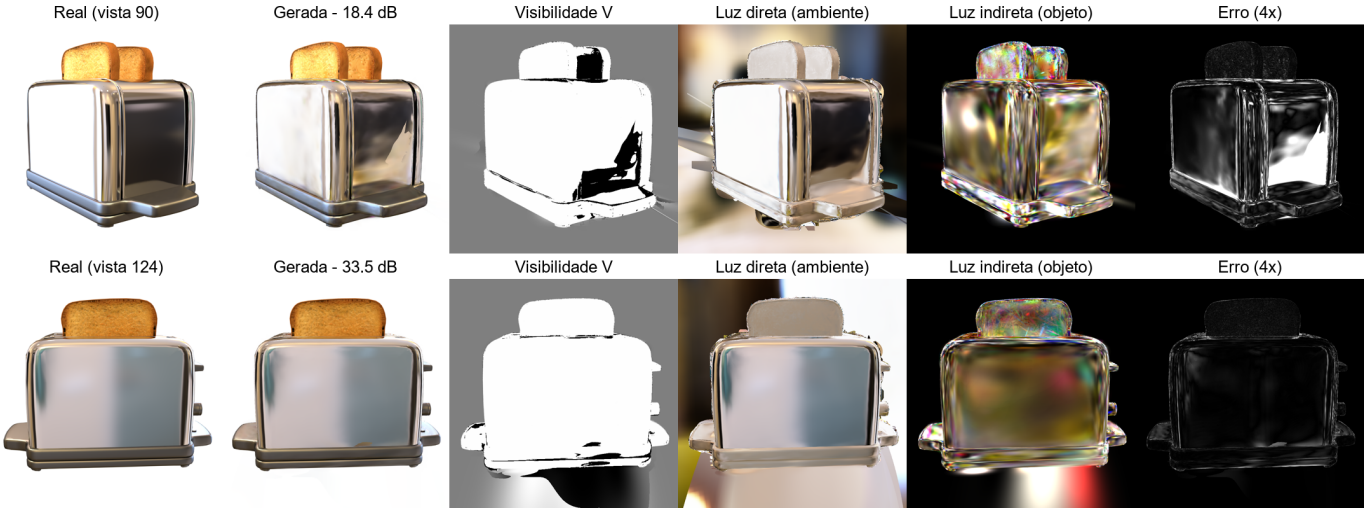


Figura 7. Mapas exportados do renderizador na pior (acima, vista 90) e na melhor (abaixo, vista 124) vistas de teste da *toaster*: imagem real, renderização, visibilidade V (preto: raio bloqueado; cinza: fundo), luz direta do mapa de ambiente, radiância indireta aprendida e erro absoluto ($4\times$).

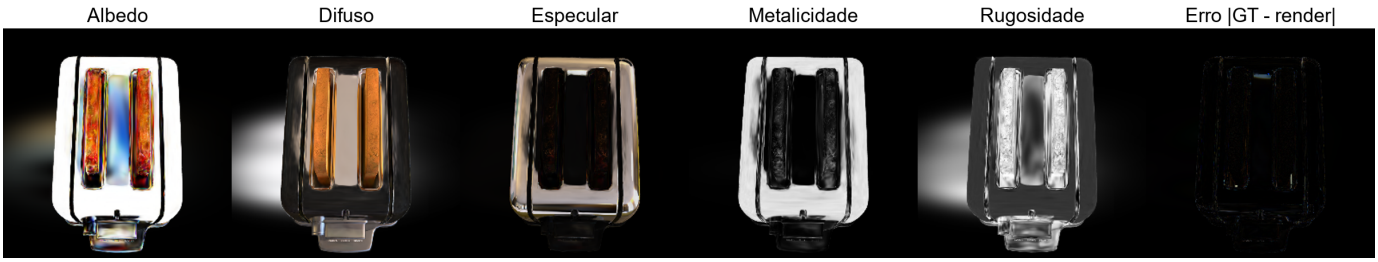


Figura 8. Mapas intermediários da renderização diferida na cena *toaster* (iteração 50.000, vista de treino): albedo, componentes difusa e especular, metalicidade, rugosidade e erro absoluto em relação à imagem real.



Figura 9. Mapa de ambiente aprendido na cena *toaster* (projeção equirretangular), reconstruído apenas a partir dos reflexos observados no objeto.

o atributo aprendido do baseline.

V. Conclusão

Este relatório apresentou o método Ref-Gaussian, adotado como baseline da dissertação, e discutiu duas de suas limitações: a visibilidade testada em uma única direção e a dependência de uma malha extraída periodicamente. Para cada uma foi proposta uma linha de investigação. O baseline foi portado para Windows e executado em uma GPU de notebook, e a replicação reproduziu os resultados do artigo nas seis cenas do Shiny Blender: SSIM médio idêntico, LPIPS a 0,001 e PSNR 0,57 dB abaixo, diferença que se reduz a poucos décimos quando se avalia o checkpoint de 40 mil iterações com a malha correspondente. A análise do mapa de visibilidade mostrou que os erros se concentram nos pixels em que o raio refletido é dado como bloqueado e que a causa principal é a radiância indireta aprendida por Gaussiana, e não o teste de oclusão. Esses resultados definem a referência para avaliar as propostas e apontam P2 como a linha mais promissora.

Referências

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[2] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian splatting for geometrically accurate radiance fields," in *Proc. ACM SIGGRAPH*, 2024.

[3] Y. Yao, Z. Zeng, C. Gu, X. Zhu, and L. Zhang, "Reflective Gaussian splatting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[4] Y. Jiang, J. Tu, Y. Liu, X. Gao, X. Long, W. Wang, and Y. Ma, "GaussianShader: 3D Gaussian splatting with shading functions for reflective surfaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[5] K. Ye, Q. Hou, and K. Zhou, "3D Gaussian splatting with deferred reflection," in *Proc. ACM SIGGRAPH*, 2024.

[6] D. Verbin, P. Hedman, B. Mildenhall, T. Zickler, J. T. Barron, and P. P. Srinivasan, "Ref-NeRF: Structured view-dependent appearance for neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.

[7] Y. Liu, P. Wang, C. Lin, X. Long, J. Wang, L. Liu, T. Komura, and W. Wang, "NeRO: Neural geometry and BRDF reconstruction of reflective objects from multiview images," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[8] B. Karis, "Real shading in Unreal Engine 4," in *SIGGRAPH Course: Physically Based Shading in Theory and Practice*, 2013.

[9] N. Moenne-Loccoz, A. Mirzaei, O. Perel, R. de Lutio, J. M. Esturo, G. State, S. Fidler, N. Sharp, and Z. Gojcic, "3D Gaussian ray tracing: Fast tracing of particle scenes," *ACM Trans. Graph.*, vol. 43, no. 6, 2024.